LAB SHEET 03

1. Set Up the React App Using Vite

```
npm create vite@latest react-todo -- --template react
cd react-todo
npm install
npm run dev
```

This creates a React application using Vite. The development server can normally be opened at the local address shown in the terminal.

2. Project Structure

```
react-todo/
├── src/
│   ├── components/
│   │   ├── AddTaskForm.jsx
│   │   ├── TaskList.jsx
│   │   └── TaskItem.jsx
│   ├── App.jsx
│   ├── App.css
│   └── main.jsx
├── index.html
└── package.json
```

3. Component Tree Diagram

```
App
│
├── AddTaskForm
│     └── input + Add button
│
└── TaskList
      ├── TaskItem
      ├── TaskItem
      └── TaskItem
```

4. App.jsx — Parent Component

```
import { useEffect, useState } from "react";
import AddTaskForm from "./components/AddTaskForm";
import TaskList from "./components/TaskList";
import "./App.css";

function App() {
  const [tasks, setTasks] = useState(() => {
    const saved = localStorage.getItem("react-tasks");
    return saved ? JSON.parse(saved) : [];
  });

  useEffect(() => {
    localStorage.setItem("react-tasks", JSON.stringify(tasks));
  }, [tasks]);

  const addTask = (text) => {
    const newTask = {
```

```
      id: Date.now(),
      text,
      completed: false
    };
    setTasks((prev) => [...prev, newTask]);
  };

  const toggleTask = (id) => {
    setTasks((prev) =>
      prev.map((task) =>
        task.id === id
          ? { ...task, completed: !task.completed }
          : task
      )
    );
  };

  const deleteTask = (id) => {
    setTasks((prev) => prev.filter((task) => task.id !== id));
  };

  return (
    <div className="app">
      <div className="todo-card">
        <h1>React Todo App</h1>
        <p className="subtitle">Component-based task management</p>

        <AddTaskForm onAddTask={addTask} />

        <TaskList
          tasks={tasks}
          onToggle={toggleTask}
          onDelete={deleteTask}
        />
      </div>
    </div>
  );
}

export default App;
```

5. AddTaskForm.jsx

```
import { useState } from "react";

function AddTaskForm({ onAddTask }) {
  const [text, setText] = useState("");
  const [error, setError] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    const value = text.trim();

    if (!value) {
      setError("Please enter a task.");
      return;
    }

    onAddTask(value);
    setText("");
    setError("");
  };
```

```
  return (
    <form onSubmit={handleSubmit} className="add-form">
      <div className="input-row">
        <input
          value={text}
          onChange={(e) => setText(e.target.value)}
          placeholder="Enter a task"
        />
        <button type="submit">Add</button>
      </div>

      {error && <p className="error">{error}</p>}
    </form>
  );
}

export default AddTaskForm;
```

6. TaskList.jsx

```
import TaskItem from "./TaskItem";

function TaskList({ tasks, onToggle, onDelete }) {
  if (tasks.length === 0) {
    return <p className="empty">No tasks yet. Add your first task!</p>;
  }

  return (
    <div className="task-list">
      {tasks.map((task) => (
        <TaskItem
          key={task.id}
          task={task}
          onToggle={onToggle}
          onDelete={onDelete}
        />
      ))}
    </div>
  );
}

export default TaskList;
```

7. TaskItem.jsx

```
function TaskItem({ task, onToggle, onDelete }) {
  return (
    <div className="task-item">
      <label>
        <input
          type="checkbox"
          checked={task.completed}
          onChange={() => onToggle(task.id)}
        />
        <span className={task.completed ? "completed" : ""}>
          {task.text}
        </span>
      </label>

      <button onClick={() => onDelete(task.id)}>
        Delete
      </button>
```

```
    </div>
  );
}

export default TaskItem;
```

8. main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

9. App.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #eef2f7;
}

.app {
  min-height: 100vh;
  display: grid;
  place-items: center;
  padding: 30px;
}

.todo-card {
  width: min(650px, 100%);
  background: white;
  padding: 30px;
  border-radius: 18px;
  box-shadow: 0 12px 35px rgba(0,0,0,.10);
}

h1 {
  margin-bottom: 5px;
}

.subtitle, .empty {
  color: #6b7280;
}

.input-row {
  display: flex;
  gap: 10px;
  margin: 25px 0 5px;
}

.input-row input {
  flex: 1;
  padding: 12px;
```

```
  border: 1px solid #d1d5db;
  border-radius: 8px;
}

button {
  border: 0;
  border-radius: 8px;
  padding: 10px 16px;
  background: #4f46e5;
  color: white;
  cursor: pointer;
}

.error {
  color: #dc2626;
}

.task-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 13px;
  margin-top: 10px;
  background: #f5f7fb;
  border-radius: 9px;
}

.completed {
  text-decoration: line-through;
  color: #9ca3af;
}

.task-item button {
  background: #dc2626;
}

@media (max-width: 600px) {
  .input-row {
    flex-direction: column;
  }
}
```

